

Technical Interview: spoofable

Project: spoofable, an SPF and DMARC email spoofing auditor **Candidate:** Alex Williams **Course:** COMP163, NC A&T, Fall 2026 **Date:** September 14, 2026

ABOUT THIS DOCUMENT

Questions were asked by Claude (Opus 5), acting as a technical interviewer on the finished project. **All answers below were written by me, in my own words, without AI assistance, and without looking at the code.** They are reproduced exactly as I first wrote them, including the ones that turned out to be wrong.

After I submitted my answers, I asked the interviewer to mark each one. Those **Assessment** blocks were written by Claude, not by me, and are clearly labeled as such. I have deliberately left my original answers unedited rather than correcting them after the fact, because the assignment treats finding out what you cannot explain as the point of the exercise.

Result: 7 correct, 6 partially correct, 2 incorrect, 2 unanswered.

THE PROJECT

1. Walk me through this project. What does it do, and why did you build it?

Answer: This project is designed to check a specific domain and see if a malicious actor can pretend to be someone registered with that domain and send email as that domain.

Assessment: Partially correct. The core function is stated accurately. Two things missing. First, the question asks *why* you built it, which is unanswered. Second, the project is a measurement study, not only a tool: it was pointed at NC's five public HBCUs and the Fortune 15, and the dataset is the actual contribution. Minor precision issue: an attacker does not need to be "registered with" the domain. They need no relationship to it at all, which is the whole problem.

2. Explain SPF to someone who has never heard of it.

Answer: SPF sets who is allowed to send mail on your behalf, usually by way of an IP address.

Assessment: Correct. Accurate and concise. Worth adding that the list is published as a DNS record, which is what makes it publicly auditable and is why this tool can read it without touching anyone's systems.

3. Explain DMARC to someone who has never heard of it.

Answer: DMARC acts as a security guard to a mail server. It operates on a series of flags (p= none, quarantine, or reject). If set up as none, then the server lets any and all mail that has a correct domain in the from: section (which can be spoofed). If set to quarantine, the server will detect whether or not there is a mismatch between the from: and where the server is actually receiving the mail from, and if there is a mismatch then the email goes to spam (still readable by the user). if set to reject, then if there is a mismatch the message does not reach the user.

Assessment: Mostly correct. The three policies are described accurately and the security guard framing works. One muddled point: under p=none the receiver still performs the mismatch check in full and then ignores the result. It is not that it skips checking. Missing the name for the mechanism being described, which is **alignment**, and missing that DMARC governs DKIM as well as SPF.

THE REASONING

4. A domain publishes a flawless SPF record, locked down, ending in -all, no mistakes anywhere. It publishes no DMARC record. Your tool labels it SP00FABLE. Defend that. Isn't SPF the thing that's supposed to stop spoofing?

Answer: You need both. DMARC is what deals with quarantining or rejecting.

Assessment: Incorrect, or more precisely, does not defend the position. The claim "you need both" is true but unsupported, and the obvious follow-up, "why do you need both?", has no answer here.

The defense turns on SPF validating the **envelope sender**, a field the recipient never sees, rather than the **From:** header, which is the only sender a mail client displays. An attacker sends from their own domain, with their own correct SPF record, and passes. What they passed was a test about a domain nobody looks at.

Noted for the record: the candidate reached exactly this conclusion unprompted during the earlier teaching session, describing the attacker as "never pretending to be something else in the backend, just presenting a different front to what the user sees," which is an accurate statement of the mechanism. It did not make it onto the page. This is the most important question on the sheet.

5. What is the practical difference between p=none, p=quarantine and p=reject for a forged message?

Answer: p=none means that the server sees there's a mismatch between what your message claims it's from vs where it actually is from, but it just dumps it in the inbox. quarantine means that the server sees there's a mismatch, but still lets it reach its destination, albeit in spam. reject means that the server sees a mismatch and prevents it from reaching the user, in any way, at all.

Assessment: Correct. All three accurately described, and the p=none case is stated better here than in Q3.

6. An organization's security dashboard shows their DMARC record as valid and correctly formatted. Can I still forge email from them? What would you need to look at to know for sure?

Answer: Depends. You need to see if it is set up as none, quarantine, or reject.

Assessment: Partially correct. Identifies the right field to check. The direct answer to the question asked is **yes**, because a record can be perfectly valid and still provide zero protection, which is the distinction the question is probing: valid is a statement about syntax, protected is a statement about policy. Two other tags also weaken a good policy and are not mentioned: `pct=`, which applies the policy to only a percentage of failing mail, and `sp=`, which can leave every subdomain open while the main domain is locked. UnitedHealth in this project's own dataset is `p=reject; sp=none` for exactly this reason.

7. Why does your tool not check DKIM?

Answer: Unknown

Assessment: Not answered. The reason is a genuine technical constraint rather than an omission: DKIM public keys are published at a DNS name built from a **selector**, such as `selector1._domainkey.example.com`. The selector is chosen by the sender and appears only in the headers of a message they have already sent. Selectors cannot be enumerated from outside, so an external audit physically cannot check DKIM without first being sent a message. This is the reason grading on DMARC is the correct design choice and not a shortcut.

DESIGN AND CORRECTNESS

8. Why did you write your own DNS client instead of using a library? What is the downside of that choice?

Answer: I will begin with the downside. The downside of this is that there is a higher chance of errors and all resolution is up to you, the creator. Writing this (If I actually wrote it myself instead of using AI) is simply an exercise in learning more about networking.

Assessment: Partially correct, and candid in a way that counts in the candidate's favor. The downside is right, and this project proved it: three separate truncation and error-handling bugs were found and fixed during development, none of which would have existed behind a mature library.

The upside is not stated. Writing it directly means **zero dependencies**, so the tool has no install step and cannot break when a package updates, and it means direct control over timeout and retry behavior, which matters when scanning many domains against rate-limited public resolvers.

The parenthetical about AI authorship is accurate and consistent with the assignment, which directs the student to have the AI build the project.

9. What is the TC flag, and why does your code care about it?

Answer: The response from the server was too large to fit over UDP, and so was cut short (truncated). The code cares about it because it means there is more data that may be important but it just can't see it.

Assessment: Correct. Accurate on both halves. The consequence could be stated more sharply: a client that ignores the flag sees zero records and reports "no SPF record," which for large domains including apple.com, microsoft.com and ncat.edu would be a confident and completely false security finding.

10. Why does a truncated response require retrying over TCP, rather than just asking again over UDP?

Answer: Because UDP has a character limit

Assessment: Partially correct. True, but it does not answer the question, which is specifically why *retrying over UDP* does not help. The answer is that the response is the same size it was a moment ago, so the failure is **structural rather than transient**: a second UDP query returns an identically truncated answer. TCP works because the response is sent with a two-byte length prefix and read as a stream, so it has no size ceiling.

11. In your code, what is the difference between "this domain has no SPF record" and "I could not retrieve this domain's SPF record"? Why does that distinction exist?

Answer: Not having a record is different from couldn't retrieve because for one, it simply does not exist. The other one *may* exist, but the tool has no way of knowing.

Assessment: Correct. The distinction is stated properly. The reason it exists could be made explicit: collapsing the two lets a network failure masquerade as a security finding, so the tool reports something false about a real organization. Two concrete instances arose in this project. First, cisco.com and wells Fargo.com were reported as having no SPF record when the queries had merely timed out. Second, berkshirehathaway.com alternated between PROTECTED and WEAK across consecutive runs, which the candidate flagged, and which traced to a transient lookup failure being graded as a weakness. Both are worth citing in an answer to this question.

12. What happens if I point your tool at a domain that does not exist? What about a domain whose nameservers are unreachable? Should those produce the same output?

Answer: when pointing at a nonexistent domain, the tool automatically returns it as "SPOOFABLE," while also saying there is no DMARC record. The same output should occur when a domain's nameservers are unreachable.

Assessment: Incorrect, and it contradicts Q11. Q11 correctly argues that a missing record and an unretrievable one are different facts. This answer then says the two should produce identical output. Placed side by side, an interviewer will ask which position the candidate actually holds.

Unreachable nameservers must produce `ERROR`, not a verdict, which is the behavior the tool was corrected to after the `berkshirehathaway.com` finding.

This answer did, however, surface a real flaw in the tool. The first half is factually accurate: a nonexistent domain currently is graded `SP00FABLE`, because the `NXDOMAIN` response yields no records and the grading logic concludes "publishes no DMARC." That is wrong on its own terms. A domain that does not exist has no mail to forge and cannot be impersonated. This was identified through the candidate's answer and is recorded as a known limitation.

13. What is the 10-lookup limit in SPF, and why can you not count those lookups by reading the record alone?

Answer: The 10-lookup limit is a hard set rule designed to prevent network spoofing. I do not know why you can't count the lookups by reading the record alone.

Assessment: Partially correct. The limit exists, and it is hard, but the purpose is misidentified. It is not an anti-spoofing measure. It bounds the amount of DNS work a receiving mail server must perform to evaluate one message. Without a cap, a maliciously constructed SPF record could force every receiver on the internet into unbounded lookups, which is a denial-of-service amplification problem.

The unanswered half: `include`: mechanisms `nest`. An included record has its own includes, and the budget of 10 is shared across the entire recursive evaluation rather than per record. So the only way to count correctly is to fetch each included record and walk the tree, which is what `spf.count_lookups()` does, with loop detection for records that include themselves.

Worth noting: this project found that `ncat.edu` uses 9 of its 10 permitted lookups.

THE DATA

14. Your results show 40% of NC public HBCUs spoofable versus 7% of the Fortune 15. What do you conclude from that? What can you not conclude from it?

Answer: I will begin by saying that you cannot conclude that HBCU's care less about or are given less support to harden their infrastructure, because we are comparing a public university to 15 of the most profitable companies in the country, if not the world. However, you can conclude that since nearly half of the public HBCU's are spoofable, there is much room for improvement. Also, while 7% is not a lot, that is still much too high for a Fortune list that small.

Assessment: Correct, and the strongest answer on this sheet. It separates what the data supports from what it does not, identifies the confound without being prompted, and resists the more dramatic causal story the numbers invite. The final observation, that 7% is still too high for a group that small and that well resourced, is a good instinct: with fifteen organizations, one failure is not noise.

This is the answer that demonstrates the candidate understands his own data rather than just its headline.

15. Your sample is five schools and fifteen companies. Is that enough to support the claim you are making?

Answer:

Assessment: Not answered. The strongest available response distinguishes two claims. The measurement itself is a **census, not a sample**: North Carolina has exactly five public HBCUs and all five were measured, so no inference from a sample is occurring and sample-size objections do not apply. The comparison between the two groups is far weaker: with a base of five, one institution changing shifts the figure by 20 percentage points, and the Fortune 15 is not a random sample of well-resourced organizations, with five of the fifteen using the same email security vendor.

LEGALITY AND REFLECTION

16. Is what this tool does legal? How is it different from port scanning those same organizations?

Answer: Yes, it merely checks if a policy is set up. Port scanning actively finds open ports and vulnerabilities and is often used as a first step in a DOS or DDOS attack. Depending on the level of port scanning, it can also overwhelm servers, potentially leading to the aforementioned DOS/DDOS attack.

Assessment: Correct. The distinction is drawn accurately. The sharpest version of the point is that this tool reads **public DNS records**, the same records every mail server on the internet reads before accepting a message, and never connects to the organization's systems at all. Port scanning sends traffic to their infrastructure. This sends traffic only to public resolvers.

17. What is the weakest part of this project, and what would you change?

Answer: Linking the underlying network logic to the actual code. I generally understand how it works, but I fail at specifics and converting it into readable code.

Assessment: Correct and candid. An accurate self-assessment, and it matches what this interview showed: the conceptual answers about SPF, DMARC and policy were considerably stronger than the answers about implementation. Naming a real weakness rather than a flattering one is the right instinct for this question.

INTERVIEWER'S SUMMARY

	Count	Questions
Correct	7	2, 5, 9, 11, 14, 16, 17
Partially correct	6	1, 3, 6, 8, 10, 13
Incorrect	2	4, 12
Not answered	2	7, 15

Pattern. The candidate is markedly stronger on what the system does and why it matters than on how the code implements it, which he identified himself in Q17 before seeing any of these marks. Conceptual questions about policy and data interpretation were answered well. Questions requiring specifics of the DNS wire format were the weakest.

Q14 is the standout. Pushing back on your own results, unprompted, is a harder skill than reciting a protocol, and it is rarer.

Q4 is the gap that matters most, because it is the reasoning the entire tool rests on, and because the candidate demonstrably understood it in conversation before failing to write it down.

One finding came out of the interview itself. The Q12 answer identified that nonexistent domains are incorrectly graded `SP00FABLE` , a real flaw in the grading logic that had not been noticed during development.